# SC2006 Software Engineering
# Lecture 17 Supplement

Conrad Watt

# Lt16 brief recap

- Testing

# Equivalence classes and boundary values

- **Equivalence classes**

  Black-box perspective (this course)
  Group inputs according to the outputs they give

  White-box perspective
  Group inputs according to which code lines / program paths are exercised

# Equivalence classes and boundary values

- **Conventions of this course - input format**

  Generally questions will be phrased as follows:

  The function under test takes N arguments:
  - arg1, which respects format 1
  - arg2, which respects format 2
  - …
  - argN, which respects format N

  Then some details about the accept/reject behaviour

# Equivalence classes and boundary values

The function isOverweight takes

- weight, an integer from 0-999
- height, an integer in the range 0-999

if weight is < 20 or > 700, reject
if height is >300, reject
otherwise, calculate (weight / (height / 100)^2 >= 25)

weight equivalence classes:
lower invalid: 0-19, valid 20-700, upper invalid 701-999

height equivalence classes:
valid 0-300, upper invalid 301-999

# Equivalence classes and boundary values

The function isOverweight takes

- weight, a 16 bit signed integer
- height, a 16 bit signed integer

if weight is < 0, reject
if height is < 0, reject
otherwise, calculate (weight / (height / 100)^2 >= 25)

weight equivalence classes:
lower invalid: -32768 - -1, valid 0-32767

height equivalence classes:
lower invalid: -32768 - -1, valid 0-32767

# Equivalence classes and boundary values

The function isOverweight takes

- weight, a 16 bit signed integer
- height, a 16 bit signed integer

if weight is < 0, reject
if height is < 0, reject
if weight > 700, return “outside human limits!”
if height > 300, return “outside human limits!”
otherwise, calculate (weight / (height / 100)^2 >= 25)

weight equivalence classes:
rejected: -32768 - -1, valid 0-700, inhuman: 701-32767

height equivalence classes:
rejected: -32768 - -1, valid 0-300, inhuman: 301-32767

# Equivalence classes and boundary values

weight equivalence classes:
rejected: -32768 - -1, valid 0-700, inhuman: 701-32767

weight boundary values:

rejected:
just below: -32769(*)
lower: -32768
upper: -1
just above: 0

valid:
just below: -1
lower: 0
upper: 700
just above: 701

inhuman:
just below: 700
lower: 701
upper: 32767
just above: 32768(*)

# Equivalence classes and boundary values

height equivalence classes:
rejected: -32768 - -1, valid 0-300, inhuman: 301-32767

height boundary values:

rejected:
just below: -32769(*)
lower: -32768
upper: -1
just above: 0

valid:
just below: -1
lower: 0
upper: 300
just above: 301

inhuman:
just below: 300
lower: 301
upper: 32767
just above: 32768(*)

# Equivalence classes and boundary values

- **boundary value testing on isOverweight**

  choice in applying the course technique, depending on whether we consider the inhuman class as being valid or invalid

  remember we test all combinations of valid boundary value inputs

  and then every combination of inputs where exactly one input comes from an invalid boundary value

# Equivalence classes and boundary values

- **boundary value testing on isOverweight**

  assuming inhuman is an invalid equivalence class…

  test all combinations of upper/lower values from both valid equivalence classes:

  valid input pairs
  0, 0
  700, 0
  0, 300
  700, 300

# Equivalence classes and boundary values

- **boundary value testing on isOverweight**

  assuming inhuman is an invalid equivalence class…

  test all combinations for upper/lower value of rejected weight equivalence class, and upper/lower value of valid height equivalence class:

  input pairs where weight is rejected (invalid)
  -32768, 0
  -1, 0
  -32768, 300
  -1, 300

# Equivalence classes and boundary values

- **boundary value testing on isOverweight**

  assuming inhuman is an invalid equivalence class…

  test all combinations for upper/lower value of inhuman weight equivalence class, and upper/lower value of valid height equivalence class:

  input pairs where weight is inhuman (invalid)
  701, 0
  32767, 0
  701, 300
  32767, 300

# Equivalence classes and boundary values

- **boundary value testing on isOverweight**

  assuming inhuman is an invalid equivalence class…

  test all combinations for upper/lower value of valid weight equivalence class, and upper/lower value of rejected height equivalence class:

  input pairs where height is rejected (invalid)
  0, -32768
  0, -1
  700, -32768
  700, -1

# Equivalence classes and boundary values

- **boundary value testing on isOverweight**

  assuming inhuman is an invalid equivalence class…

  test all combinations for upper/lower value of valid weight equivalence class, and upper/lower value of inhuman height equivalence class:

  input pairs where height is inhuman (invalid)
  0, 301
  0, 32767
  700, 301
  700, 32767

# Equivalence classes and boundary values

- **boundary value testing on isOverweight**

  assuming inhuman is an invalid equivalence class…

  because inhuman is an invalid equivalence classes, don't write any tests where both weight and height are inhuman

# Equivalence classes and boundary values

- Boundary value testing only makes sense for numeric ranges

- General concept of equivalence classes is useful for other forms of data (e.g. Enum, Objects, etc)

- The course tells us to only test one invalid input at a time. This is a good start, but real test suites will also include at least a few tests with multiple invalid inputs
  - test priority of different error messages

# Equivalence classes and boundary values

The function isOverweight takes

- weight, a 16 bit signed integer
- height, a 16 bit signed integer

if weight is < 0, reject
if height is < 0, reject
if weight > 700, return “outside human limits!”
if height > 300, return “outside human limits!”
otherwise, calculate (weight / (height / 100)^2 >= 25)

weight equivalence classes:
rejected: -32768 - -1, valid 0-700, inhuman: 701-32767

height equivalence classes:
rejected: -32768 - -1, valid 0-300, inhuman: 301-32767

# Equivalence classes and boundary values

- Many types are complicated enough that we want to pick multiple representatives from each equivalence class

  example:

  insert_into_sorted_tree(Tree t, Node n)

  t: a Tree
  n: a Node

  if t is not sorted
  (left child nodes < parent, right child nodes > parent)
  then **reject**

# Equivalence classes and boundary values

- insert_into_sorted_tree(Tree t, Node n)

  t: a Tree
  n: a Node

  if t is not sorted
  (left child nodes < parent, right child nodes > parent)
  then **reject**

- clearly t has two equivalence classes
- n appears to have one equivalence class

- does this mean that two tests based on equivalence class testing are enough???